

9/16/2019

Internship Report

Implementation of an application for monitoring data and performance.

Sonoo Viloshna

Innovative Information Systems

University of Toulouse Capitole

viloshna-luxmi.sonoo@ut-capitole.fr

Supervisor

Mr Aziz Mahbub

Amazon.com

muaziz@amazon.com

University Tutor

Mr Umberto Grandi

University of Toulouse Capitole

umberto.grandi@ut-capitole.fr



PREFACE AND ACKNOWLEDGEMENTS

For five and a half months from April until September 2019, I did an internship at Amazon.com, the world's largest online retailer and prominent cloud services provider. I was part of the Amazon Europe team and was based in DTM1 Fulfilment Center in Werne, Germany. This internship project is a part of my 2-year master program at the University of Toulouse Capitole in France.

I worked on a data monitoring tool, the Heimdall portal. Heimdall is a platform that pulls information from several sources, processes the data and displays the data on a web interface with the primary objective of monitoring the performance of the team and the different services provided. In this report, I will emphasize on the objectives and the different aspects of the Heimdall portal.

The topic of the internship is in line with the 2IS master program since it allowed me to work with some new innovative technologies and techniques. Throughout my tenure at Amazon, I did not only improve my technical skills but more importantly, I also had the opportunity to sharpen my soft skills in a professional working environment.

I would like to express my gratitude to my supervisor, Mr Mahbub Aziz for his guidance during this internship and for giving me opportunities to push my limits and explore new horizons. I would also like to thank all my lecturers at UT1 for their continuous support during those two years. The courses taught have equipped me with the skills and knowledge to complete this internship. Special thanks goes to my academic supervisor Mr Umberto Grandi, not only for his guidance during this internship but also for always being present to advise and motivate me during the last two years.

TABLE OF CONTENTS

Table of Figures	IV
Glossary of terms	VI
1. Introduction	1
Objectives	1
Scope of work.....	2
About Amazon.....	2
Fulfilment Center.....	2
ICQA.....	3
2. Background.....	4
3. Project Management.....	8
Project Planning	8
Software Development Approach	9
Contribution and Responsibilities.....	10
4. Requirements	11
Needs analysis	11
Overview of Heimdall.....	13
5. Architecture Design.....	16
Interaction with other applications.....	16
Heimdall Portal Architecture	18
6. Implementation	19
Scheduled Rake Tasks.....	19

ETL Job and Lambda	23
Web application	24
Models - Active Records	24
Controllers	24
Views	25
Challenges and solutions	36
7. Assessment	39
Assessment of Project.....	39
Skills Acquired.....	40
How did the 2IS Master program help during this internship?	41
Future Works.....	42
8. Conclusion	42
9. References	43

TABLE OF FIGURES

Figure 1 Generalised from SQS white paper example [4].....	7
Figure 2 SIM Sprint Board Example	9
Figure 3 Automated build and release through pipeline.....	10
Figure 4 Architecture Diagram - Interaction with other systems.....	16
Figure 5 Architecture Diagram - Heimdall Application	18
Figure 6 BPMN Workflow - SIM Synchronisation	20
Figure 7 BPMN Workflow - User Visits Logs Import	21
Figure 8 BPMN diagram for Redshift Queries Import	23
Figure 9 View to display SIM Details	26
Figure 10 SIM Dashboard	27
Figure 11 Sprint Monitoring Dashboards.	28
Figure 12 Chart showing progress of each team member	28
Figure 13 HeatMap showing open job failures	29
Figure 14 Breakdown of Job failures by client and user	30
Figure 15 Data Table displaying queries based on their query and execution times.....	30
Figure 16 Charts showing metrics on query execution times.....	31
Figure 17 User Visits Dashboard.....	32
Figure 18 Page to view and run rules.....	33
Figure 19 Creation of a SIM issue when a rule fails	33
Figure 20 Creation of a new rule	34
Figure 21 Datable containing portal details	34

Figure 22 View to consult Teams and Users details	35
Figure 23 View to display and edit database connections	35
Figure 24 Display and create new Datanet alarms	36
Figure 25 Execution time without optimisation	37
Figure 26 Execution time with hash map optimisation	37
Figure 27 Job Failure Trend over the past two months.....	39
Table 1 Table of Glossary	VI

GLOSSARY OF TERMS

Term	Meaning
Associates	The employees of Amazon are referred to as associates
BPMN	Business Process Modelling Notation
DJS	Distributed Job Scheduler
ETL	Extract, Transform, Load
EU	Europe
FC	Fulfilment Center
ICQA	Inventory Control and Quality Assurance
LDAP	Lightweight Directory Access Protocol
SIM	Simple Issue Management
SNS	Simple Notification System
SQS	Simple Queue System

Table 1 Table of Glossary

1. INTRODUCTION

Every day, we generate gigabytes of data at Amazon Europe alone. There are multiple internal applications which have been developed to process those data and provide reports to facilitate the jobs of the associates. Those applications must be efficient, available and reliable in order to increase productivity and customer experience.

It is essential to have a monitoring system to track and optimise the performance of our applications. A good monitoring tool also helps to detect anomalies and prevent serious incidents which saves time and money.

During my internship, I was assigned to work on the Heimdall monitoring tool. The main aim of Heimdall is to ensure that we deliver top quality services to our users. It provides metrics about the different internal portals(applications) such that any anomaly can be easily detected. Heimdall portal is a web application which extracts data from various sources, processes the data and provides user friendly dashboards which help the developers to monitor the applications. The interface is a web application implemented using Ruby on Rails.

In this report, I will introduce the company, the team and the objectives of the project. I will also give a brief explanation of the architecture of the application and different AWS services involved. The report also highlights the needs for such an application. In the Implementation section, I will give a more in depth idea of how the systems work. In the last parts of the report, the challenges and assessment of the project will be covered.

OBJECTIVES

- I. Notify developers by creating tickets when a job results in error.
- II. Increase visibility of job failures through an interactive heatmap.
- III. Implement a dashboard to monitor and analyse the team's tickets in real time in order to enhance the process of ticket resolution.
- IV. Visualise the key performance indicators to track and evaluate the team's performance.
- V. Monitor the load on the redshift cluster and detect any anomaly.
- VI. Define data control strategies to allow users to create rules to check data consistency, correctness and completeness.

SCOPE OF WORK

The Heimdall platform has been developed mainly for the EU ICQA Analysts team and their portals. However, the platform is scalable and has been designed in such a way that it can be extended to other teams by configuring the necessary parameters and minimal changes to the code.

ABOUT AMAZON

Founded in 1994 by Jeff Bezos, Amazon was formerly known as an online book selling platform but has since established itself as a retail giant. Amazon offers a plethora of services but is most known for ecommerce and its cloud solutions. By its revenue and market value, Amazon has become the largest internet retailer in the world. In 2019 Amazon became the world's most valuable brand at \$315.5 billion. It is considered one of the big fours among top technological companies (Apple, Google & Facebook).

The success behind Amazon can only be attributed to its workforce, cutting-edge technological infrastructure and 14 leadership principles [1]:

- Customer Obsession
- Ownership
- Invent and Simplify
- Are Right, A Lot
- Learn and Be Curious
- Hire and Develop the Best
- Insist on the Highest Standards
- Think Big
- Bias for Action
- Frugality
- Earn Trust
- Dive Deep
- Have Backbone; Disagree and Commit
- Deliver Results

FULFILMENT CENTER

Amazon Fulfillment Centers (FC) are large facilities with hundreds of employees, sometimes thousands. Employees are called associates. Each FC are categorised by site type depending

on the type of products they handled. FC activities are divided into two namely inbound and outbound.

Inbound

Inbound associates carry out two main types of tasks.:

- Receiving and unloading of goods.
- Inspecting the goods and stowing which refers to placing the goods in storage and recording their location.

Outbound

Outbound associates are on the other hand responsible for the following three tasks:

- Picking up goods from shelves.
- Sorting and Packing of goods.
- Loading of packages on trucks for shipping.

Amazon's fulfilment network is one of the most advanced in the world. With its pioneering FBA service, (Fulfilment By Amazon) Amazon has changed the business landscape for many entrepreneurs. Sellers can now store their products at Amazon's warehouses to then be handled by Amazon for packing, shipping and customer service.

Amazon Logistics Werne GmbH (DTM1) is one of the many fulfilment centers of Amazon that have contributed to this growth and the increasing needs of their expanding worldwide customer base.

ICQA

ICQA stands for Inventory Control and Quality Assurance. Inventory Control is a system of accounting for items designed to increase efficiency, safeguard inventory, and avoid error, loss and damage. Quality Assurance is a system of researching and preventing errors before they happen in order to provide the best level of quality service to customers.

The ICQA mission is to decrease defects in Amazon's inventory through continuous process improvements to meet and exceed customers' expectations, both buyers and sellers, with defect-free order completion. That is, customers get exactly what was ordered in the best possible condition because the product was processed in the most effective and accurate way achievable.

2. BACKGROUND

In this section, I will cover some concepts and technologies which are used directly or indirectly in this project and which are required to have a better understanding of the project.

Simple Issue Management

Simple Issue Management (SIM) is an issue and project tracking tool which is used to manage tickets(bugs), backlogs, and sprints.

An issue is the core object type in SIM. It corresponds to a unit of work that needs to be completed. It may be any type or size of work, such as a Customer Contact, Backlog item task/story/epic, Feature Request, Work Order, Auto-cut Alarm, Security Campaign. Issues may have sub-tasks that further break down components of the work to be completed.

Issues are divided into two categories:

- Tickets: to indicate a problem a bug which requires a resolver's intervention.
- Simple Issues: Simple issues are usually tasks which need to be completed as part of a project backlog.

SIM also allows users to run and track agile projects by defining sprints and linking tasks to sprints. Moreover, developers can also estimate the points need to complete a task, input the number of points spent and the number of points remaining.

The SIM portal can also be access through a REST API. Although the SIM API can be accessed directly using HTTP, there are a number of libraries that have been developed for different languages such as JAVA, Python and Ruby.

Redshift Cluster

Amazon Redshift is a fully managed, petabyte-scale data warehouse service in the cloud. It uses its own Redshift relational database and provides an SQL based client interface matching industry standard of JDBC and ODBC drivers for PostgreSQL [3]. However, contrary to regular RDBMS, Redshift internally stores the data as a columnar database. This is an optimisation for fast retrieval of data, analytics and ideal for big data processing (parallelism).

To note that Redshift manages the translation from row-based data management /querying to column-based storage seamlessly without the need for user interference. A Redshift cluster

consists of a leader node and compute nodes. The leader node acts as a gateway to requests while compute nodes handle queries. The columnar storage and clustering abilities of Redshift provide the scalability potential required for growing data needs. Redshift provides bulk queries to facilitate migration from other databases. It goes without saying that migrating from relational databases such as PostgreSQL or MySQL might only require minor or no changes.

Datanet

Datanet is Amazon's internal Big Data Management System and is the gateway to the Data Warehouse. DataNet has a number of web applications that facilitate the process of getting data in and out of the Data Warehouse. Two of the most important ones used in this project are:

- ETL Manager (ETLM) - The core "operating system" of the Data Warehouse. This complex system manages all of the injection and extraction of data in a consistent and predictable way. Users interact with ETLM through the DataNet web application by scheduling load or extract jobs and defining where to retrieve or pull the data.
- Klaxon - Klaxon is DataNet's intelligent alarming system for ETLM Jobs. These alarms can notify you via emails or tickets when data will arrive late or if a job is in error.

DJS - Distributed Job Scheduler

As its name suggests, the DJS is a platform to schedule background jobs (unix commands or scripts). The job specifications which consists of the Apollo environment, command to be executed, variables and scheduling time and interval can be configured on the user interface. The jobs can be also linked to monitors to generate notifications upon failure of the job.

Athena

Amazon Athena is built on Presto, an open source distributed SQL query engine for big data. Athena is serverless and runs when queries are called. It allows the querying directly to Amazon S3 using SQL queries and a predefined schema without the need to setup Extract, Transform and Load (ETL) processes. Athena also automatically runs queries in parallel where possible reducing overall execution time. [5]

Glue

As mentioned with Athena a predefined schema of the data is required to query the data. In comes Amazon Glue; a fully managed ETL service which can be used to catalogue, clean and

enrich data stores. The Data Catalogue contains table definitions, job definitions, and other control information to help you manage your AWS Glue environment. [6]

Hammerstone

Hammerstone is a data integration platform designed to accelerate the internal adoption of Amazon Redshift, AWS's data warehouse. Hammerstone provides a user-friendly UI which allows customers to create workflows and activities designed to extract, load and transform data. Hammerstone allows users to use Redshift without having to develop, test, and maintain ETL scripts, queries, and workflows. It also offers features like the scheduler, precondition, alarm, monitoring, source-control, ad-hoc query, cluster-management, pipeline management, etc...

CloudWatch

Amazon CloudWatch is a monitoring service for AWS cloud resources and the applications that run on AWS. It can be used to collect and track metrics, collect and monitor log files, set alarms, and automatically react to changes in the AWS resources. It can be used to gain system-wide visibility into resource utilization, application performance, and operational health.

Simple Queue System (SQS)

Amazon Simple Queue Service is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications [2]. Using SQS one can, for example, ensure that long running batch jobs be handled in a non-blocking asynchronous way to the user. A status of the long running job can be shown to the user while the load is handled by multiple virtual machines (for example EC2) through request and response queues.

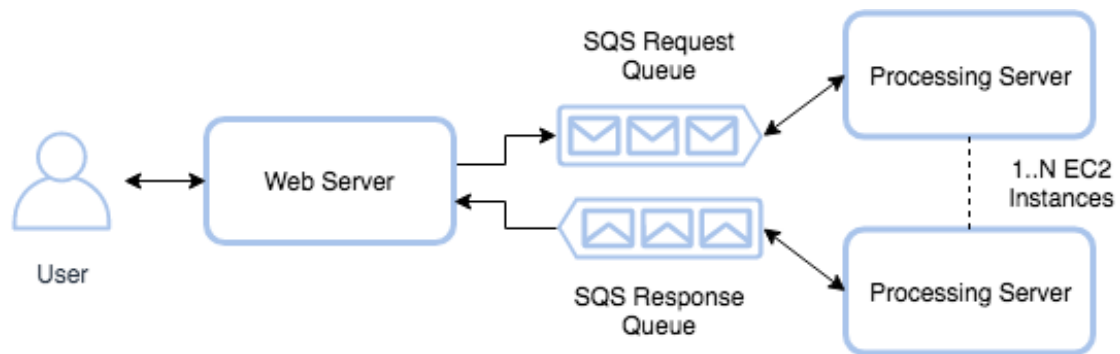


Figure 1 Generalised from SQS white paper example [4]

SQS offers locking mechanisms on the queues and First-In-First-Out ordering if required among many other features. This allows the possibility for different robust availability and even staged-event-driven architectures.

AWS Lambda

AWS Lambda, most commonly referred to as Lambda, is a serverless computing service that runs a piece of code in response to triggers such as changes in data, shifts in system state, or actions by users[9]. It can also be triggered by AWS services such as changes in Amazon S3 buckets and table updates in DynamoDB, SNS, Cloudwatch. This allows the setting up of real-time serverless data processing systems. Lambda supports multiple runtimes including Java, Python, Node.js and Ruby.

Pipelines

Pipelines is a tool to model and automate the release process. Pipelines tool helps to achieve continuous deployment. Continuous deployment is a development methodology where every software change is automatically built, tested and pushed to production immediately when the code is submitted by the developer. AWS Pipelines can be easily integrated with third-party services such as GitHub.

ICQA Portals

ICQA portals refer to the different applications developed and maintained by the EU ICQA Analysts team. Some examples include Home, Repo, Pacman, Transhipment Portal and Star. These applications help associates in their tasks by providing reliable metrics and reports. We will not go through each application in depth because it is not required for this project.

3. PROJECT MANAGEMENT

PROJECT PLANNING

The EU ICQA Analysts team consists of 14 software developers and analysts who are based in different Fulfilment Centers across Europe. The EU ICQA analyst team is focused on supporting the Europe ICQA Network by creating simple and effective dashboard reporting to provide greater visibility, facilitate the tasks of the EU ICQA managers and team leads, identify and drive process improvements and find opportunities to reduce defects.

Since, the team members are not in the physical location, an adapted scrum framework is used. The team meetings are conducted through online calls using Amazon Chime.

The team manages several projects and there can be one or more developers working on one project. The Heimdall project consisted of 3 team members, the project manager and two interns. Sprints are two weeks long. Daily scrum meetings are scheduled between Heimdall project team members to discuss about the progress, challenges and upcoming tasks. These meetings last around 15 to 20 minutes.

Each Tuesday, the whole EU ICQA Analysts team participates in a one-hour meeting known as the “Tech Call”. The urgent issues are discussed, and everyone is also invited to give overview and/or quick demo of their respective projects. There are also occasionally presentations on some interesting topics such as machine learning.

The project tracking tools used is called the Simple Issue Management (SIM). The Sim platforms allows users to create tasks and subtasks, estimate the number of points to complete a task, assign tasks to developers and move tasks along the sprint board. The sprint board for the team consists of the following lanes: Open, Processing, Implementation, Review, Release and Done. Developers can also track the points spent on a task, the remaining points required to finish the task.

The team's tasks are divided into two categories namely backlogs and issues. Backlog refers to the new features to be developed and issues to bug in existing projects escalated from end users. A SIM folder is created for each project and tasks are created in the corresponding folder.

Task points are estimated in a Fibonacci format: 1,2,3,5,8,13,20. Burn down method is used to monitor the amount of work remaining. It is the responsibility of each developer to estimate

the task points and update the points spent and remaining. The SIM platform provides a burn up and burn down chart to monitor the team's progress.

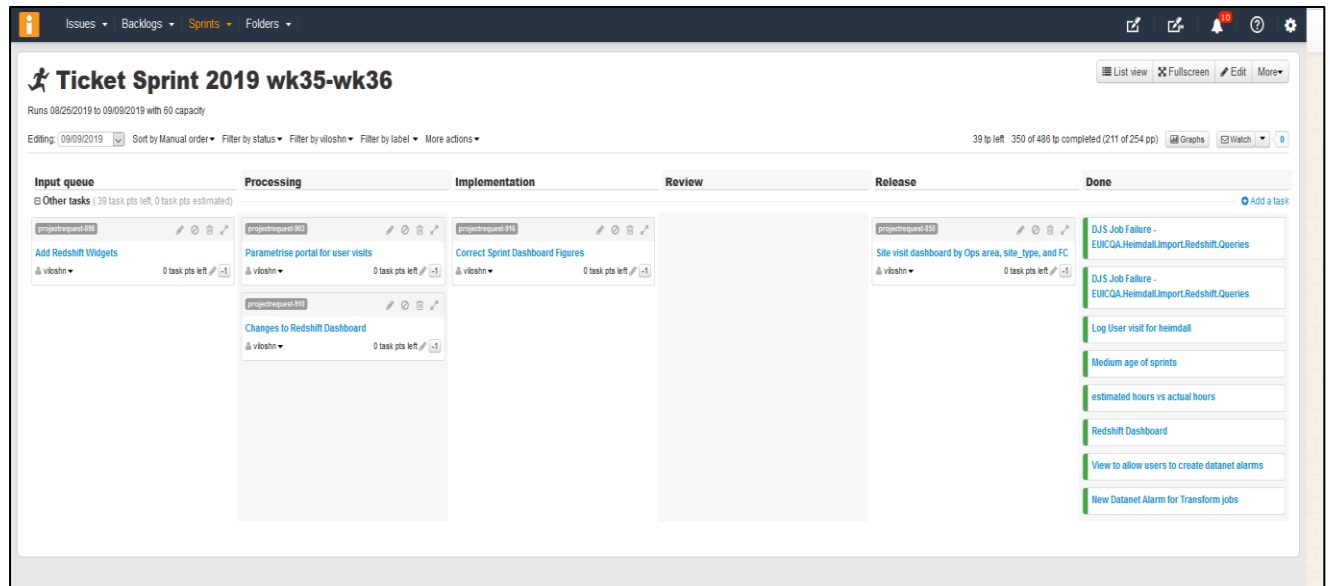


Figure 2 SIM Sprint Board Example

SOFTWARE DEVELOPMENT APPROACH

As mentioned in the project planning part, an agile methodology was used for software development. An iterative and incremental approach was followed while developing the application in order to cater for changing requirements and new features.

The application was broken down in sub modules, SIM Dashboard, User Visit dashboard, Job Failure monitoring and Redshift Queue Management. The modules were developed independently and integrated to the main application. Working software was released as soon as they were validated. For example, the SIM dashboard was developed and pushed to production first. The users already started using this functionality long before the Redshift Queue Management was released.

During the weekly Tech Calls, short demos were organised to present the product and get the feedback of the team members who are also the end users. After receiving their inputs and requests, the changes were implemented and released as an increment. Changing requirements were welcomed even during later stages of development and after release.

A lot of importance was also given to simplicity, reusability and efficiency of codes. As far as possible, we were encouraged to implement components in such a way so that they are reusable and therefore maximizing the amount of work not done in accordance with agile software development principles.

Continuous deployment was used to release new piece of working software as soon as they are available. When a code is pushed on Git, the revision just has to be approved on AWS pipelines and the changes are promoted to production, built and deployed automatically. This methodology is time saving and provides users with fixes, updates and new features early and often.

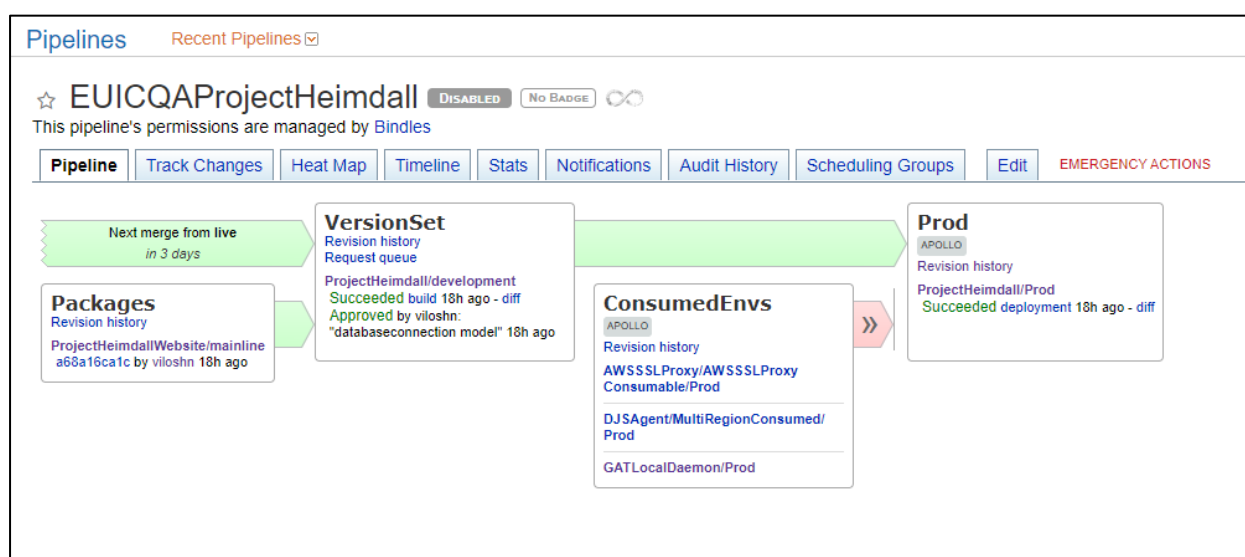


Figure 3 Automated build and release through pipeline

CONTRIBUTION AND RESPONSIBILITIES

As a software development intern, my core mission in this project was to implement and deliver an efficient and reliable working software. I performed the following tasks during the internship.

- I used the ETL Manager to write complex SQL Queries to extract data from the data warehouse and publish the data into file.
- I wrote ruby scripts to extract data from multiple sources (Redshift cluster and MySQL databases) and load the data into the Heimdall database.
- I created MySQL tables through Ruby Active model migration.

- I also worked on the implementation of the web application both server side processing and frontend (using bootstrap).
- I was responsible to configure alarms for Datanet and DJS job. I also wrote scripts to create automated alarms for future jobs.
- I worked on the Lambda function and connecting it to our database.
- Finally, at the end of the internship, I contributed to a wiki document to guide users through the portal.

In short, I have worked on all the scripts and modules covered in this report except the SIM Synchronisation script, which was done in collaboration with another intern.

4. REQUIREMENTS

NEEDS ANALYSIS

SIM

SIM is the ticket tracking platform. While the SIM platform is quite advanced and is a good tool for logging and tracking tickets, it does not cater for some of the team's requirements especially team performance monitoring. It does not provide metrics such as the number of tickets created and resolved over a period, the number of tickets based on their severity or the number of tickets resolved by each developer. These figures are necessary to evaluate the performance of the team and it helps to detect any anomaly or pending tickets.

There is an existing project which pulls data from the SIM platform and uses tableau to breakdown and represent the data. However, those charts are generated and forwarded to the team on a weekly basis and they are not interactive.

Redshift Queue Management

Everyday around 70,000 to 80,000 queries are executed on the team's redshift cluster. These include some scheduled ETL queries as well as ad-hoc user queries. It is crucial that the ETL jobs run on schedule and quickly to provide timely report to the users.

However, the jobs are dependent on a number of resources that include tables and other jobs, for example a job can extract data from a table only when after another job has updated the data in that particular table. This frequently results in prolonged waiting times. Even though

the execution time is very fast, there is a long queue time that negatively affects the performance. Sometimes the system can even experience bottlenecks. There is also the issue of inefficient queries which consume a lot of resources and have long execution time. These factors affect the delivery of reports to end users.

It is important to monitor the usage of the redshift cluster and the performance of the queries in order to identify the conflicting resources and inefficient queries to optimise the jobs and acquire better performance.

Currently, there exists a table that contains the list of queries and their execution details (queue time, execution time, database user etc.). A report is manually extracted everyday by one of the team members and presented in the Tech Call. If that team member is on leave, the data is not extracted. Moreover, that table is updated daily and the old data is purged. Hence, there is no history available for analysis and comparison.

The need is to have an interactive dashboard to view data for the last week to compare the performance of queries and detect anomalies and identify jobs which require optimisation.

Job Failures

The team manages thousands of jobs on DJS, Athena, Datanet, glue and Hammerstone. The jobs are prone to failures due to numerous causes such as resource unavailability, unexpected data format, incorrectly handled exceptions among others. Every day, on average 20 job failures are raised.

To be able to quickly and efficiently fix the errors, there should be a proper notification system to alert the developers. Services like DJS, Datanet and Athena already have their own alarm systems. However, not all jobs are linked to an alarm.

Furthermore, good visibility of job failures is required to identify the jobs which are resulting in errors, to quickly resolve those issues. It is also important to fix to the root causes of the issues to prevent the failures from occurring, leading to more stable, reliable and sustainable applications.

User Visit Logs

The team manages five portals, which are accessed mainly by managers, and leads from the EU ICQA team. Each time a user visits a portal, an entry is logged in the portal's database for

marketing purposes and to have a visibility. Currently, the Heimdall application contains a view which displays the user visit logs data for each portal for comparison.

However, this module is very slow since each time the page is accessed, Heimdall will connect to the databases of each portal and retrieve the required data. Each day around 10,000 user visits are logged for the five portals. It takes around 2-3 minutes to load the page which is not optimised at all. Moreover, the module has been developed in such a way that if a new portal goes live, the Heimdall code must be modified to include the new portal.

There is a need to optimise the existing platform to make room for scalability. Firstly, to speed up the loading of the dashboard and secondly to facilitate the integration of new portals without having to modify the code.

OVERVIEW OF HEIMDALL

The Heimdall portal has been developed to address the above needs. Heimdall is a web application with the purpose of ensuring a high standard and quality of our different software (portals) in terms of performance, response time to address issues and functionalities provided.

Heimdall platform is divided in four modules: SIM dashboard, Job Failure Dashboard, User Visit Dashboard and Redshift Queue Management.

SIM Module

The SIM module contains several interactive charts which shows the number of open and closed tickets, the ages of tickets, the most recent and the oldest tickets, the number of tickets by severity and the number of tickets by projects and by developer. The dashboard is updated in real time with a latency of 10 – 15 minutes that is each time a ticket is created or updated on SIM platform, Heimdall is also updated.

A sprint dashboard is also available. This chart breaks shows the total estimated tasks points, total effort burnt, remaining tasks points, number of tasks completed, median age of tickets for each sprint and each developer.

These dashboards help both the developers and the manager to monitor and evaluate the performance of the team and to have an overview of which tickets need to be tackled in priority.

An advantage of Heimdall is that the database table containing the SIM tickets are only updated when there is a change on the SIM platform whereas in the previous tableau

dashboard, all the tickets were retrieved and updated at regular intervals. Hence, there is an optimisation of resource consumption.

Job Failure Module

The job failure dashboard shows three charts. The first chart is an interactive heatmap displaying the open job failures grouped by client or by assignee over time. The other two are pie charts depicting the percentage of job failure for each client and number of failure tickets resolved by each developer. By drilling down on the pie chart, we can view the list of jobs aggregated by their failure rate. The importance of this module is to emphasize on which jobs are failing most often and need to be investigated in order to contribute to the smooth running of the different applications.

User Visit Module

The User Visit page displays the number of visits and number of users who accessed the different portals over one week. The location (FC) of the user is also fetched when retrieving the data. Therefore, we can also view the number of user visits per operational area, FC and site type. This gives an overview of which FCs have most active user on our portals and identify whether users from other FCs have some trouble using the applications and need some training. This data is not live. It is updated daily. Records more than one week old are archived.

Redshift Queue Management Module

The Redshift Queue Management page displays the queries having the highest queue, execution or total (queue + execution) times for the last seven days. It also contains a line chart which displays the mean queue and execution times and a boxplot chart which describes the trend for the week. Filters such as database user, execution time range and job id are available to narrow the results.

Data Control Strategies Module

The module allows users to create their own rules once their system is on boarded. The rules are SQL Queries. The database, input variables and output are dynamic. The application executes the queries and checks the results. If the results match, the run is successful. Else a SIM is created to notify the owner of the rule. Some examples of rules include checking if a new parameter has been added to a table or the values of a field in two databases are the same. The module also allows users to compare records from two tables or databases.

Admin Module

The admin part has been developed to allow users to add, modify and update parameters. It provides the following functions:

- Add, modify and delete a team.
- Add, modify and delete a developer.
- Bulk import developer details from redshift employees table.
- Create alarms for Datanet jobs.
- Add new portals for user visit module.
- Add and edit database connections to other portals.
- View and import sprint list form SIM.

5. ARCHITECTURE DESIGN

INTERACTION WITH OTHER APPLICATIONS

As mentioned above, the Heimdall platform retrieves data from several projects and data sources. The diagram below depicts how the Heimdall project interacts with different systems.

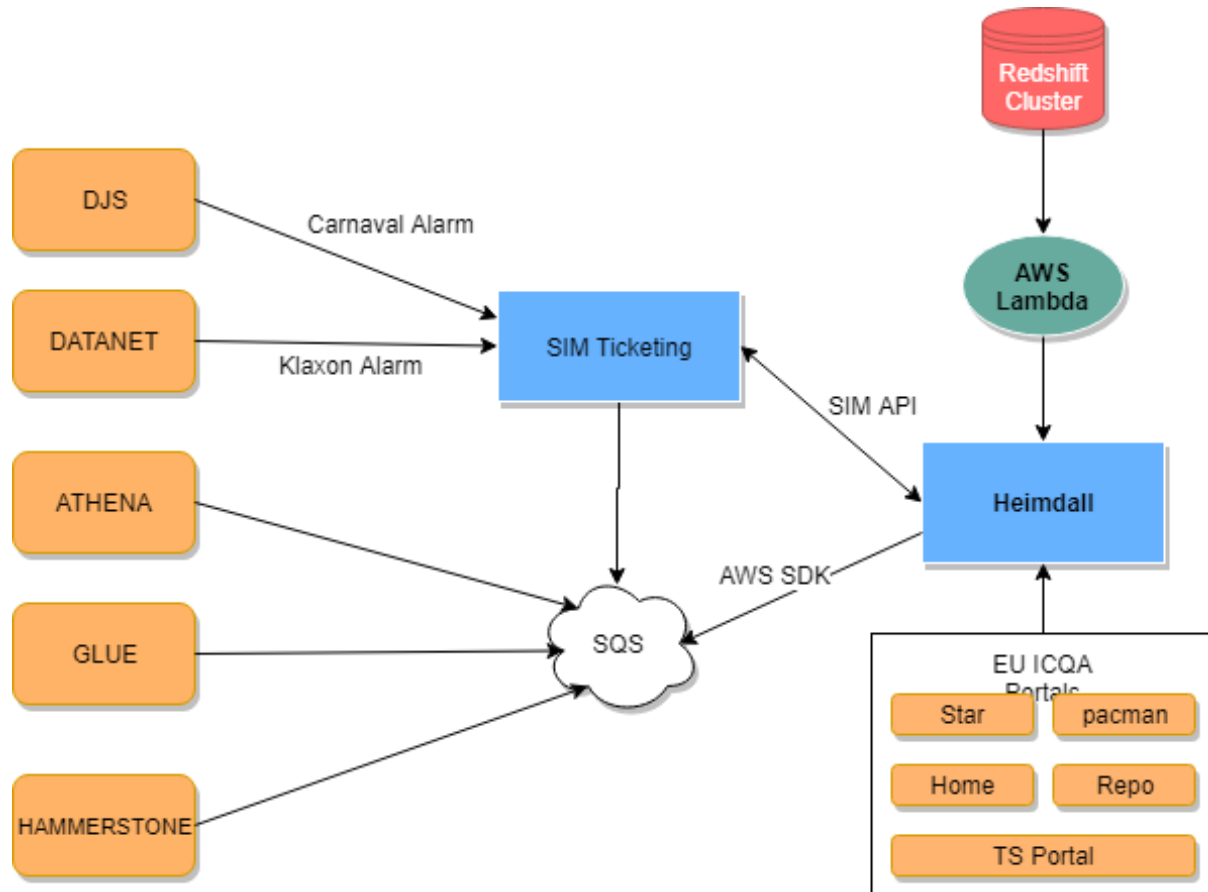


Figure 4 Architecture Diagram - Interaction with other systems

a) Scheduled jobs and Queries

Whenever a failure occurs, an issue will be created on the SIM platform to notify team members so that they can take the necessary actions. The method by which a SIM is created differs depending on the client.

Athena, Glue, Hammerstone

Athena, Glue and Hammerstone use the same process to create SIMs. Whenever, a job fails, the client publishes a message to an SNS topic. The subscriber, an SQS queue receives and

stores the message. A script which runs at regular intervals will then retrieve messages from the SQS queue and create a SIM for each message via the SIM API.

Datanet

The ETL jobs are linked to klaxon alarms which create a SIM when a job is in error state after execution. This is an inbuilt feature of the Datanet warehouse. The Datanet API is used to link new jobs to an alarm.

DJS

Carnaval alarms which are powered by CloudWatch, will trigger the creation of a SIM Ticket upon failure of a job. Same as Datanet, the DJS and Carnaval APIs are used to automatically create new monitors for new jobs.

b) SIM Platform

The EU ICQA Analysts team's folder on SIM platform is subscribed to an SNS topic. Each time a ticket is created or modified on SIM platform, a message is sent to an SQS queue. Heimdall regularly checks the SQS queue for new messages. If new messages are present, Heimdall will query SIM platform using the SIM API to retrieve the details of the impacted tickets. The Heimdall tables will then be updated.

The advantage of using SNS notification is that the Heimdall platform is synchronised in almost real time. As soon as a change is done, we know which SIM has been modified and it can be imported into Heimdall. Furthermore, unnecessary API calls are avoided since we only retrieve SIMs when a message is present in the SQS.

c) Redshift Cluster

An ETL job extracts the records from the redshift table and dumps the data in an S3 bucket. Since live data is not required, the ETL job is scheduled to run once daily. A lambda function is triggered and loads the data into the heimdall database. The rationale of using a lambda function over a scheduled script is to ensure that data is loaded as soon as it is available.

d) EU ICQA Portals

Heimdall project connects to the database of the different portals to retrieve the user visits logs on a daily basis through simple MySQL connection.

HEIMDALL PORTAL ARCHITECTURE

The Heimdall Application is built on Rails framework and has an MVC architecture.

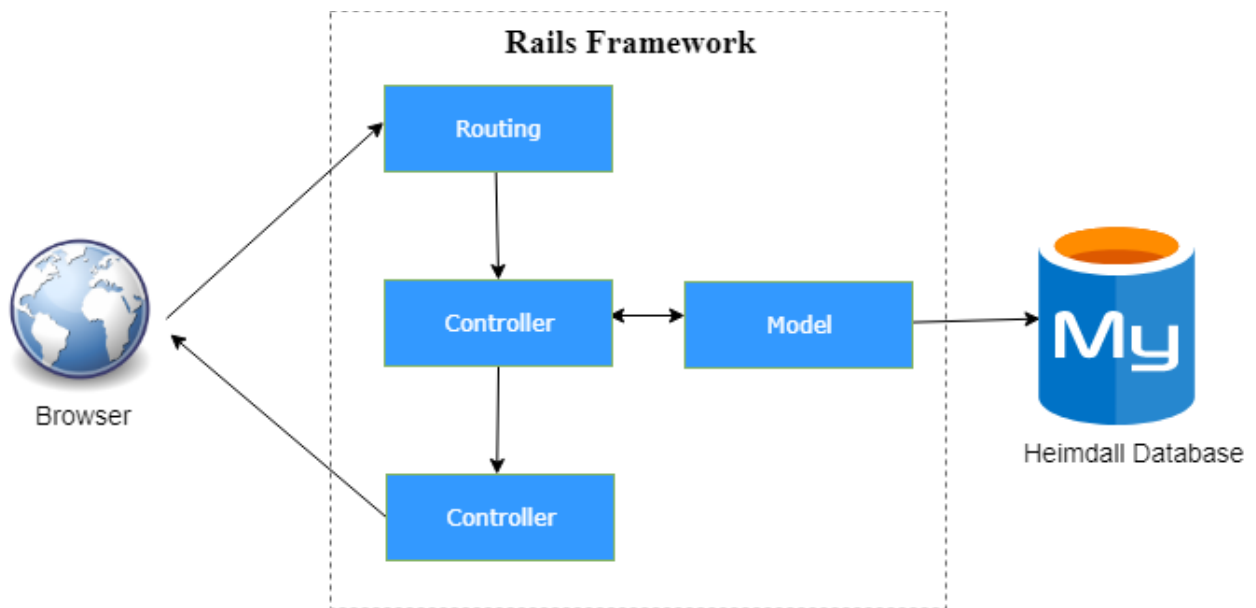


Figure 5 Architecture Diagram - Heimdall Application

a) Database

The data layer is an Amazon RDS MySQL DB instance. It is easy to operate and scalable.

b) Controller

The controller is where all the application logic happens. The model is queried for data and the data is eventually processed by the controller (in Ruby) to be sent to the view.

c) Model – Active Record

Database objects are modelled into Ruby Classes using the active record pattern. This provides better code readability, security for example against SQL injection and data management in terms of querying. Various optimisations had to be done in order to decrease the execution time of queries.

d) View

The goal of the view was to provide a dashboard. Bootstrap was used as a layout framework to cater for different screen sizes. The charts, which included a wide variety of graphs ranging

from line charts to box plots, were done using Highcharts; a library specialised in data visualisation. JQuery was used as a utility library and to make asynchronous requests to the controller as with handling big data many of the requests have an execution time between 1 to 10 seconds.

e) Routing

The routing file in Ruby maps the urls to controller actions.

6. IMPLEMENTATION

In this chapter, I will cover some technical aspect of the project so that you can get a better overview of the project. BPMN diagrams have been used to illustrate some more complex tasks. The views section, I will show some screenshots of the work done.

SCHEDULED RAKE TASKS

Rake is a native tool for Ruby, similar to UNIX's "make" [7]. Rake tasks are scripts written in ruby and launched through command line. The scripts have ".rake" extensions. In Heimdall application, rake scripts were used mainly for background jobs such as extracting data and updating the database. In this section, I will highlight some of the main rake tasks implemented.

Synchronization with SIM platform.

As mentioned in the architecture design section, whenever a SIM is created or modified, a message is sent to the SQS queue. A rake task is used to retrieve messages from the SQS queue. Each message contains the id of the SIM. If a message id found, the rake task will fetch the SIM details by querying the SIM Rest API using the id. The response is then converted to JSON format and the Heimdall MySQL table is updated. However, the complexity lies in retrieving the sprint and folder name of the SIM issue. This information is not stored in the response received from the issues/get method. We perform two additional get requests to the SIM API. Moreover, whenever a job failure ticket is received, some special processing is done to extract the job name, date run etc. These are stored in a separate table.

This rake task is also used to insert an automated comment on the SIM platform whenever a new SIM is created to inform the user that their request will be tackled as soon as possible.

The sim rake task is scheduled on DJS to be executed every 5 minutes. Figure 6 below demonstrates the flow.

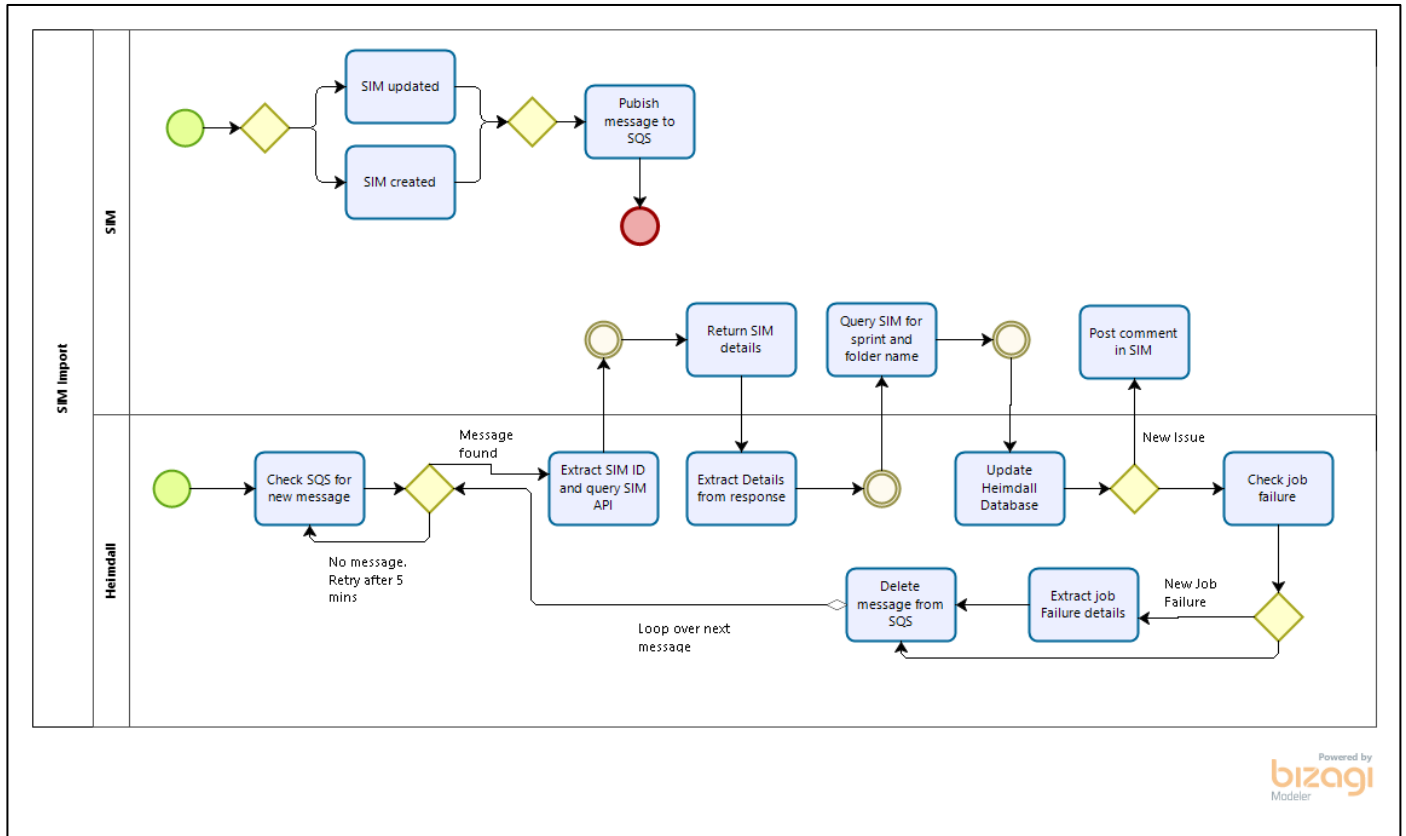


Figure 6 BPMN Workflow - SIM Synchronisation

Job Failure Notification

A rake script is also used to create a SIM ticket for an Athena, Glue or Hammerstone job failure. Similar to the sim task, the messages will be retrieved from the SQS queue and an issue will be created on SIM platform through the SIM API post method.

DJS and Datanet Alarms Synchronization Scripts.

DJS and Datanet services have their own integrated failure monitoring systems. However, when a new job is created or scheduled, the alarms are not updated automatically. Hence, the use of these scripts. The scripts get the Ids of all the jobs in the group and the list of jobs linked to the alarm. I then check which jobs are not linked to the alarm and update the latter

accordingly. Since the ids of jobs are incremental, as an optimisation, during each execution, the id of the last job found is stored. When going through the list of jobs, the script stops at the last id of the previous execution instead of looping through the whole list.

This task is scheduled to run once daily.

Import of User Visits Logs

This rake tasks connects to the different EU ICQA portals, retrieves the user visits logs. The portals' information (database connections details, table name etc.) is parametrised in an SQL table. A new portal can be easily added through the UI. The user can even add constraints to omit some records. The task builds the SQL queries based on the parameters. The choice database client is also dynamic (MySQL, PostgreSQL, redshift cluster etc.).

After retrieving the records from each portal, I retrieve the location (FC) of each user using the LDAP API. The records are then bulk inserted Heimdall database using a gem called active-record import. This saves times as the script performs only one database call to insert all records instead of one call for each record.

Since live data is not required, the task is scheduled on DJS to run daily.

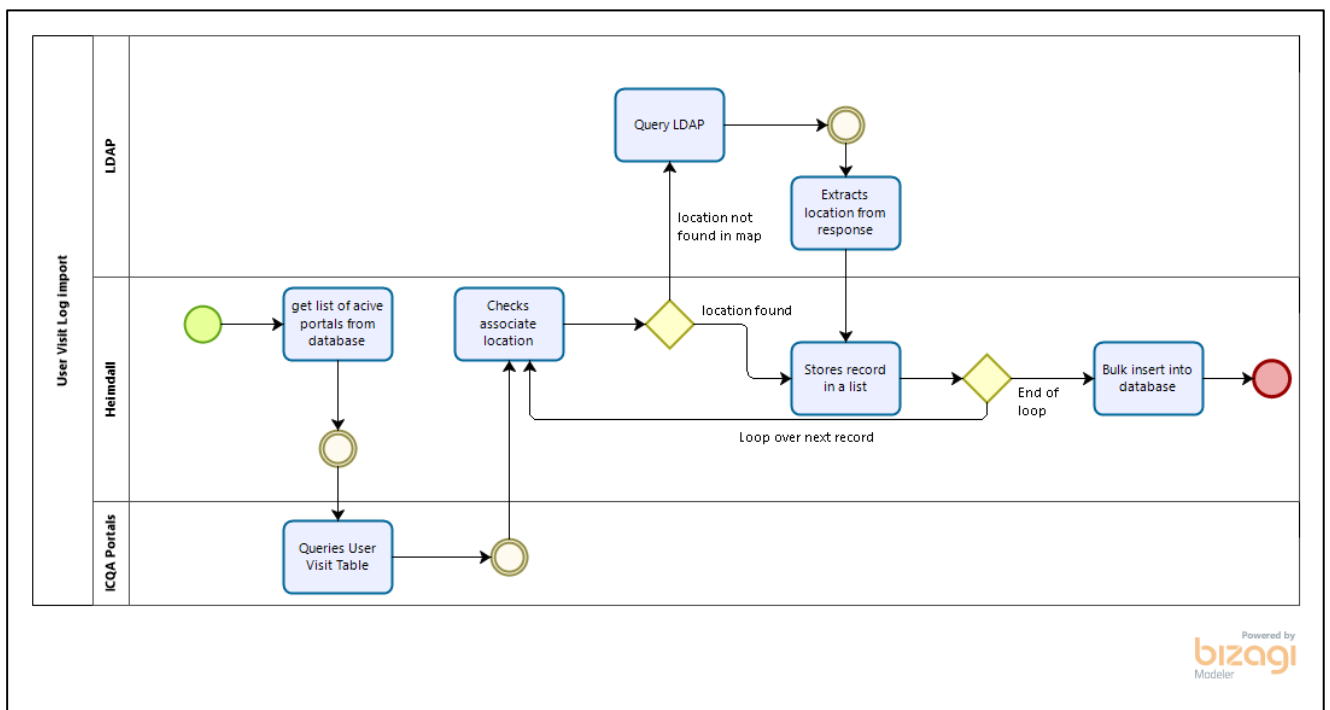


Figure 7 BPMN Workflow - User Visits Logs Import

This rake task connects to the different EU ICQA portals, retrieves the user visits logs. The portals' information (database connections details, table name etc) is parametrised in an SQL table. A new portal can be easily added through the UI. The user can even add constraints to omit some records. The task builds the SQL queries based on the parameters. The choice of database client is also dynamic (MySQL, PostgreSQL, redshift cluster etc).

After retrieving the records from each portal, each user is matched with their location (FC) using the LDAP API. The records are then bulk inserted into the Heimdall database using a gem called active-record import. This saves time as the script performs only one database call to insert all records instead of one call for each record.

Since live data is not required, the task is scheduled on DJS to run daily.

Import Users

The SIM, Sprint and Redshift Queue Management Modules offer the "Users" filter. Users refer to the team members. I created a rake task to bulk import developers from redshift as it is not practical to manually add the users one by one. The rake task loops through each team and retrieves the manager id. It then queries the redshift table and gets all users whose supervisor_id is equal to the manager_id. For all those users, I pluck the employee id and query the redshift table using the IN operator to avoid multiple calls to the redshift cluster. I again retrieve the users whose supervisor_id matches the list. This process continues until no result is returned. It usually takes 3 queries (loops) to return all the employees. This means 3 hierarchy levels. This task is not scheduled but invoked ad hoc by users.

ETL JOB AND LAMBDA

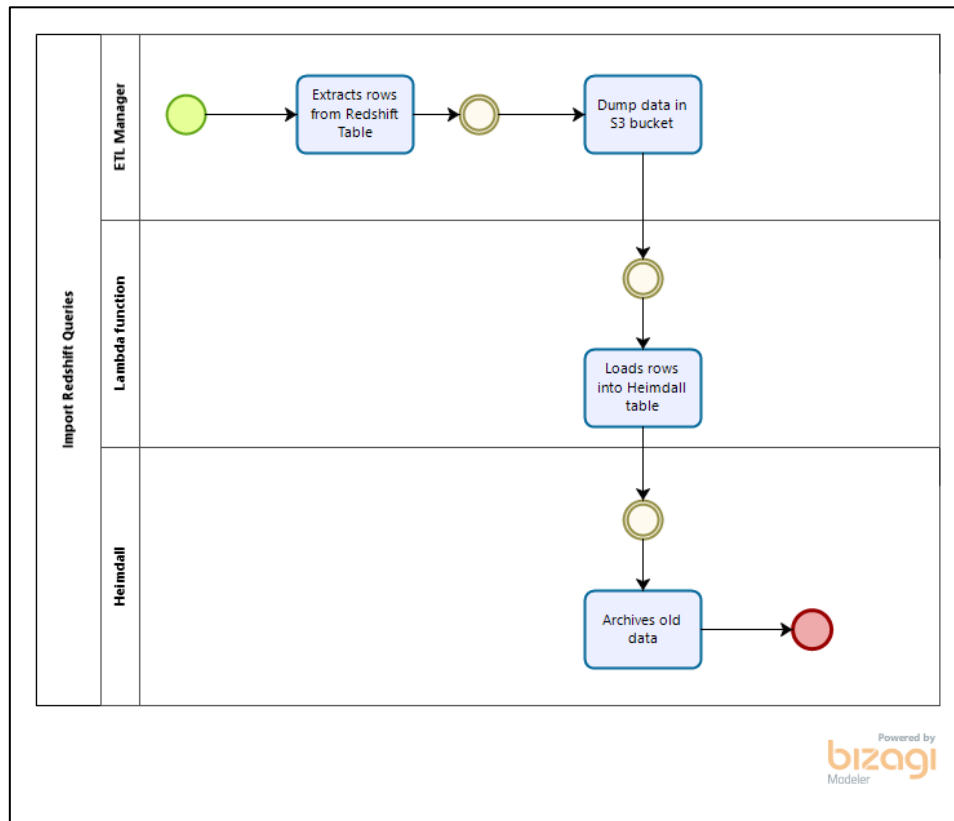


Figure 8 BPMN diagram for Redshift Queries Import

ETL operations from the Redshift cluster can be performed through the ETL Manager. The latter allows users to define profiles which are basically SQL queries. The queries can also contain virtual temp table creation. The results can then be saved in a file for either loading by another job or for processing by another external application. In our application, the file is dumped to an S3 bucket to be loaded into our database.

This project uses a lambda function to load data from an S3 file to a Heimdall database table. The lambda is triggered whenever a file is added to the S3 bucket. The trigger is an inbuilt functionality offered by AWS S3.

The lambda function is a code written in python. AWS Lambda is serverless and is independent of any application. The function is coded in such a way that you can reuse it for to load data in any database. You just have to add the connection configurations to the RDS config layer file and pass the required parameters to the function.

WEB APPLICATION

Ruby on Rails

Ruby on Rails is an open-source web application framework that uses the MCV pattern. The Heimdall application is built on Rails 5, the latest version available.

Ruby on rails contains ready-made plugins and modules which saves developers a considerable amount of implementation time. Moreover, it is very easy to integrate Amazon APIs as well as third party libraries such as bootstrap to a Ruby application. The gem dependencies have to be added to the config file and the Ruby will download the required files.[10]

On top of these advantages, Ruby on Rails was chosen because the team has been using it for some years and hence, it will be easier to maintain the application in the future.

MODELS - ACTIVE RECORDS

The model is the layer of the system responsible for representing business data and logic. Active Record gives an object-oriented aspect to the application and contributes to simpler CRUD operations and reduction in code. Active Record facilitates the creation and use of business objects whose data requires persistent storage to a database. It is an implementation of the Active Record pattern which itself is a description of an Object Relational Mapping system. [8]

Most models in Ruby on rails is mapped to a database table. The tables were created by migration that is we first generated the model and run a command to migrate the changes to MySQL database. Each change on the database was through active records migration. This ensures that our database and the models are coherent.

CONTROLLERS

The controllers do all the processing and are responsible for fetching, manipulating and updating data in the database and sending an output to the view. Parameters can be passed on to the controllers through either HTTP get or HTTP post methods. The type of HTTP request should be mentioned in the routing file. The controller sends a response to the view which is normally rendered in JSON format.

Ruby controllers have some standard methods, get, new, create, destroy and update which perform CRUD operations on active records models. However, users can define their own methods as well.

While implementing the dashboard, in most cases, the data had to first be processed by controller to produce an output accepted by Highcharts. Moreover, in some cases, the aggregations were quite complex and could not be efficiently done in an SQL query. Therefore, the raw data or partly aggregated data was retrieved and processed by the controller.

One such example is the Sprint dashboard figures. The fields for estimated points, remaining points and efforts burnt are stored in JSON format and hence cannot be aggregated at in SQL. Moreover, for the estimated points, I had to ensure that the latest estimate is being used. The rows had to be fetched. The controller checked if the data is a valid JSON object, then parse it into an array. Only then the aggregation could be done and output in a Highchart format. To achieve this, I used several data structures such as maps and sets.

VIEWS

The user interface has been implemented using HTML, HAML, JQuery and Bootstrap.

Haml is a markup language mostly used with Ruby that cleanly and simply describes the HTML of any web document without the use of inline code. It aims to reduce repetition in your markup by closing tags for you based on the structure of the indents in your code. The result is code that is well-structured, DRY, logical, and easier to read[12].

Although most of the processing is done on the server side, JQuery is used for some client-side processing and validations. It is also used for asynchronous request to the controller through AJAX calls.

Finally, Bootstrap is used as the frontend framework and Highcharts library for data visualisations. Designing data visualisations is of great importance in data analysis. It is a challenge to convey the necessary useful information to the users without overwhelming them. The flow between the charts needs to be fluent and needs tell a story to help the user in analysing the data and help in making deductions. A well-designed dashboard should be coherent and comprehensible to both an amateur and an expert data analyst alike.

At Amazon, we are encouraged to make our codes reusable and DRY. I implemented some templates known as “partials” in Ruby to breakdown the code in manageable and reusable components such as data tables, forms and functions to draw charts. The partial templates just have to be called or rendered as it is most commonly known in Ruby in the HTML pages. Parameters can also be passed to the template to customise the component. For example, for the data table, the URL, title, column names, search option, sort fields etc are sent to the partial and the table is generated automatically.

The following screenshots show some of the views implemented during the project.

SIM Module

The screenshot below shows a view where users can search for Issues based on a number of filters. The table is implemented using Data table plugin and the locally developed data table partial.

Title	Status	Requester	Submitter	Assignee	Date Create	Folder	Priority	Root Cause	Label	Sprint
"DXB3" warehouse_id not included	Closed	rasubash	rasubash	muaziz	2019-04-28 08:38:10	EU-ICQA-Analysts/EU ICQA Ticket Queue		no issue	Ticket Sprint 2019 wk17-wk18	Ticket S
"problem-receive" status	Resolved	cenders	cenders		2018-06-29 17:28:53	EU-ICQA-Analysts/EU ICQA Ticket Queue				
"SMALL & LIGHT" FC implementation CSI	Resolved	michaeli	michaeli	michaeli	2018-08-03 06:03:10	EU-ICQA-Analysts/EU ICQA Ticket Queue	0		Ticket Sprint 2018 wk30-wk31	Ticket S
"Welcome to ICQA" learning map	Open	lenzs	lenzs		2016-10-11 12:41:10	EU-ICQA-Analysts/projects/New project idee			new project request	
(EUKS) SRC not generated after Amnesty prc	Resolved	sapuna	sapuna	baumgarc	2018-05-23 08:54:21	EU-ICQA-Analysts/EU ICQA Ticket Queue		no issue	Ticket Sprint 2018 wk22-wk23	Ticket S
01 IRDR DPMO per FC per week	Resolved	toclarke	muaziz	toclarke	2018-10-15 12:36:29	EU-ICQA-Analysts			Ticket Sprint 2018 wk40-wk41	Ticket S
03 Total number of TRUE Shorts per FC per	Resolved	toclarke	muaziz		2018-10-15 12:38:53	EU-ICQA-Analysts/EU ICQA Ticket Queue			Ticket Sprint 2018 wk40-wk41	Ticket S

Figure 9 View to display SIM Details

Users can also view some interesting statistics and metric to track the current progress on the SIM Dashboard view shown below. The dashboard is interactive, and users can drilldown on the graphs to filter results. Clicking on the links in the 10 oldest and 10 newest issues table will redirect the user to the SIM platform whereas clicking on AA issues and Issues By project table will forward the user towards the SIM details view so that he can view all the SIMs for the chosen filters.

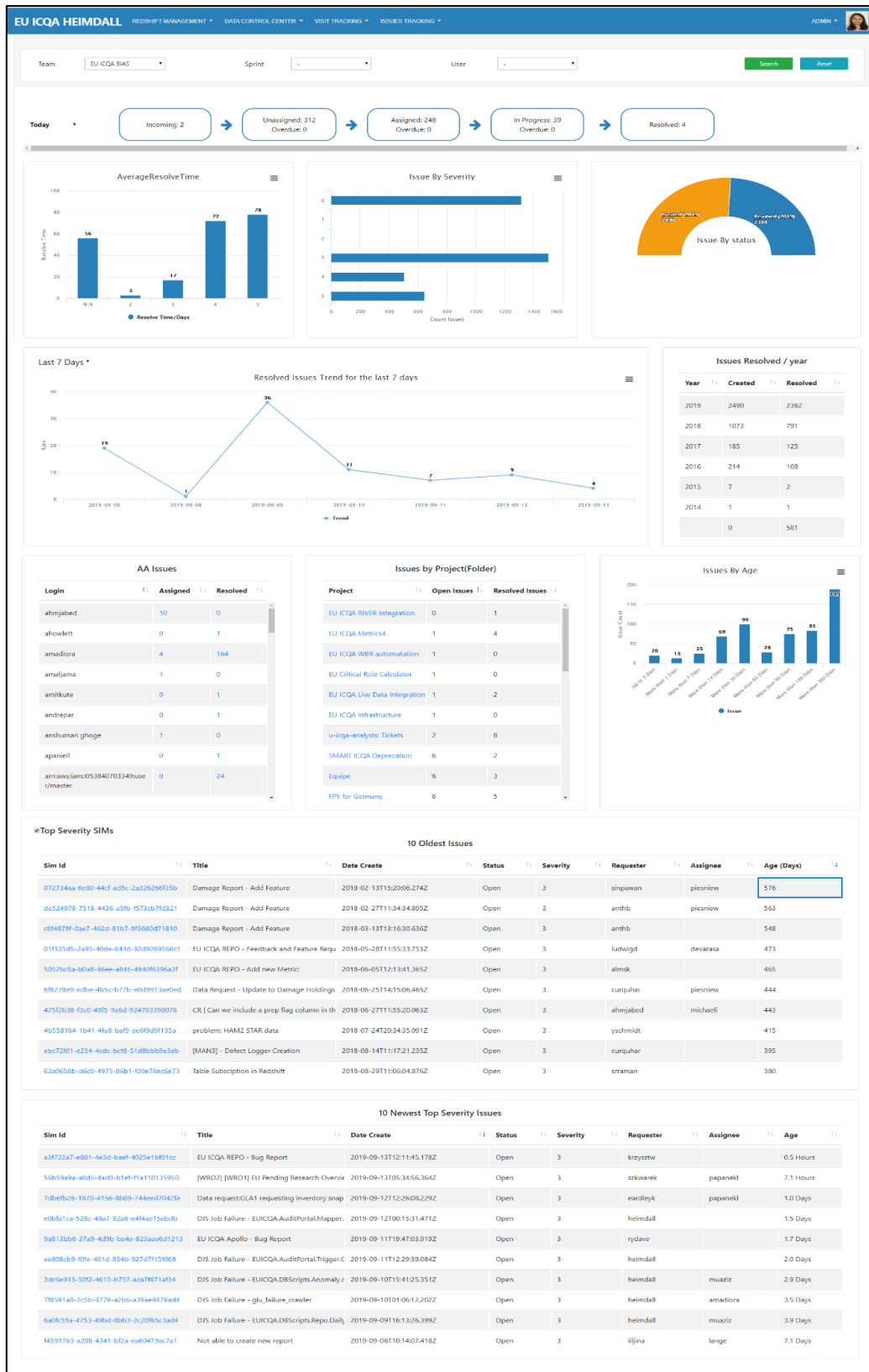


Figure 10 SIM Dashboard

Sprint Module

The Dashboards below monitors the progress of sprints. The first chart shows the number of open and completed tasks for each sprint, the estimated points, the actual completed points, the effort spent and the remaining points left to be completed.

The second line chart shows the median and mean age of tickets for each sprint.

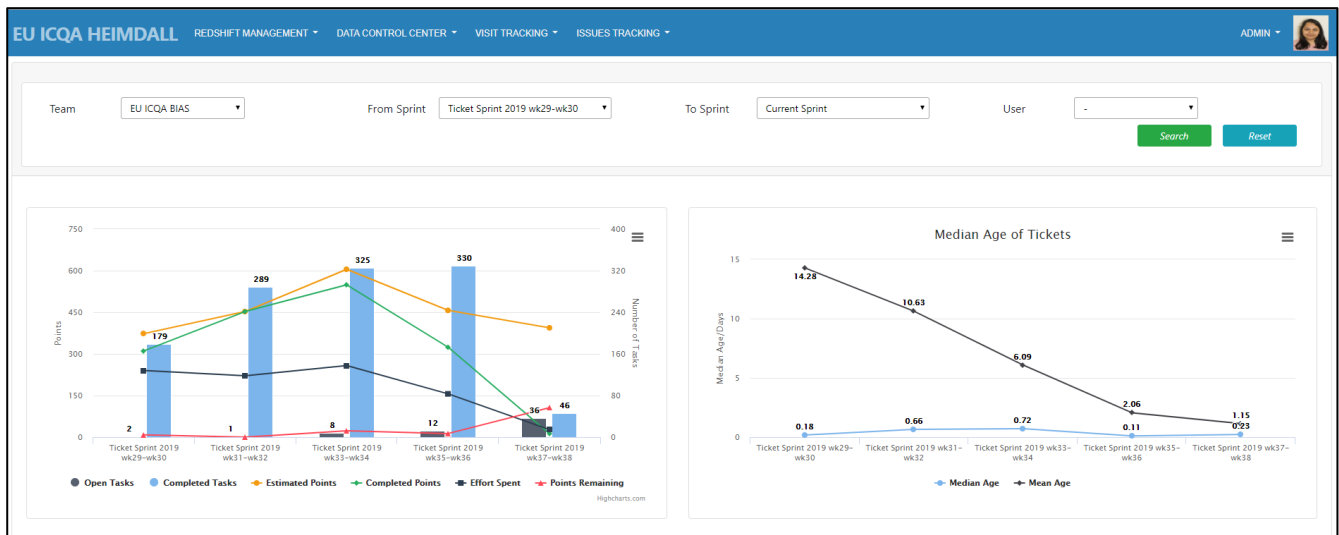


Figure 11 Sprint Monitoring Dashboards.

The dual axis chart below allows users to monitor progress of each team member for a chosen sprint. It is the same as the sprint dashboard above but breaks down the figures for each developer and it includes a percentage goal achieved metric.

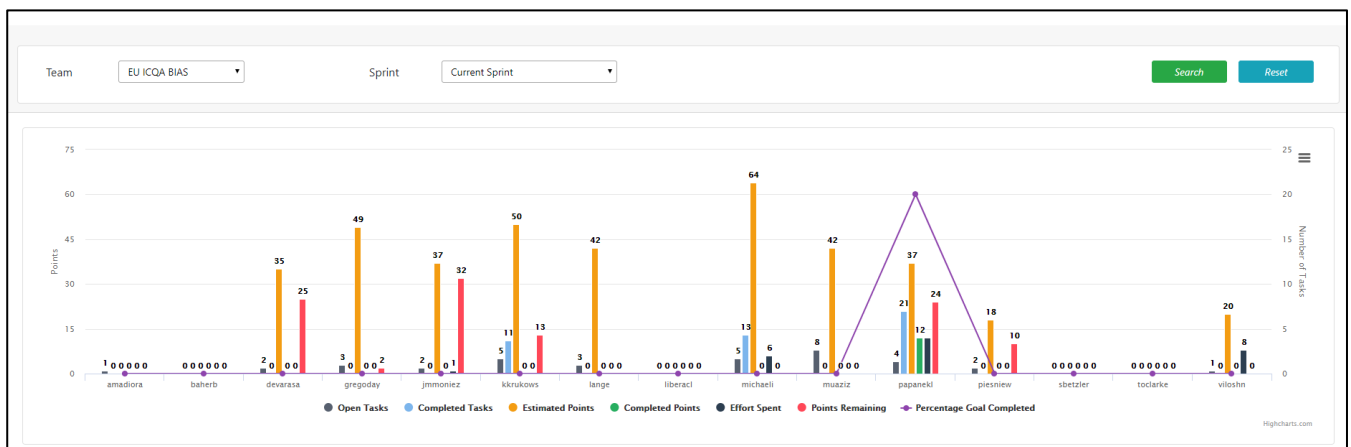


Figure 12 Chart showing progress of each team member

Job Failures

The charts below give visibility on job failures so that developers can act promptly. Users can drilldown on any grid to view the details of the job failures. Heatmap is used to focus the attention of users to the important figures at one glance.

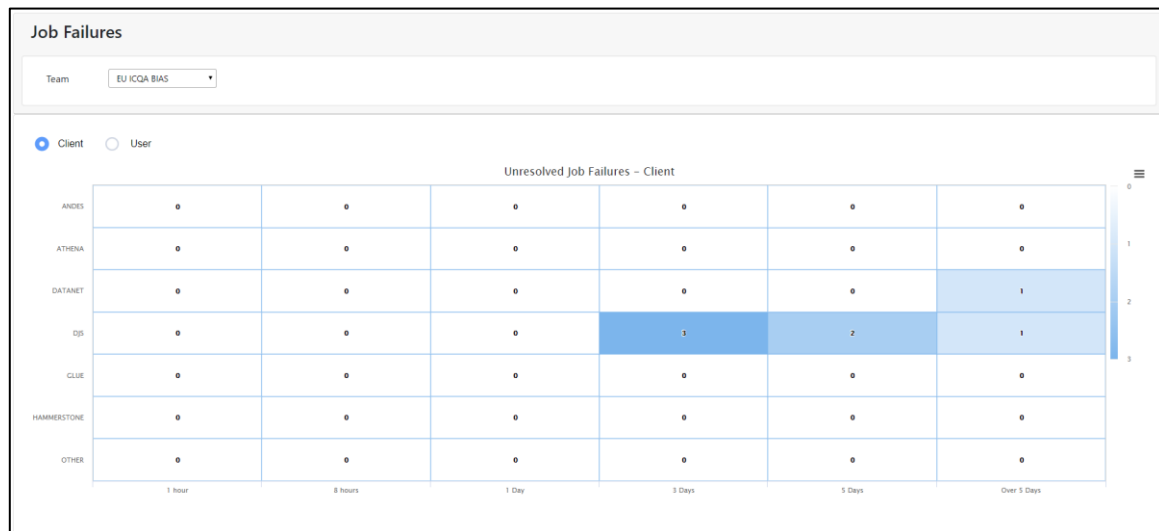


Figure 13 HeatMap showing open job failures

The screenshot below shows two pie charts aggregating the number of job failures by the client and the number of job failures resolved by each developer. The user can choose to view data for the last 7 days, last month or last year. The table shows the number of job failures grouped by Job name. It is useful to know which job is failing more in order to fix the root cause.

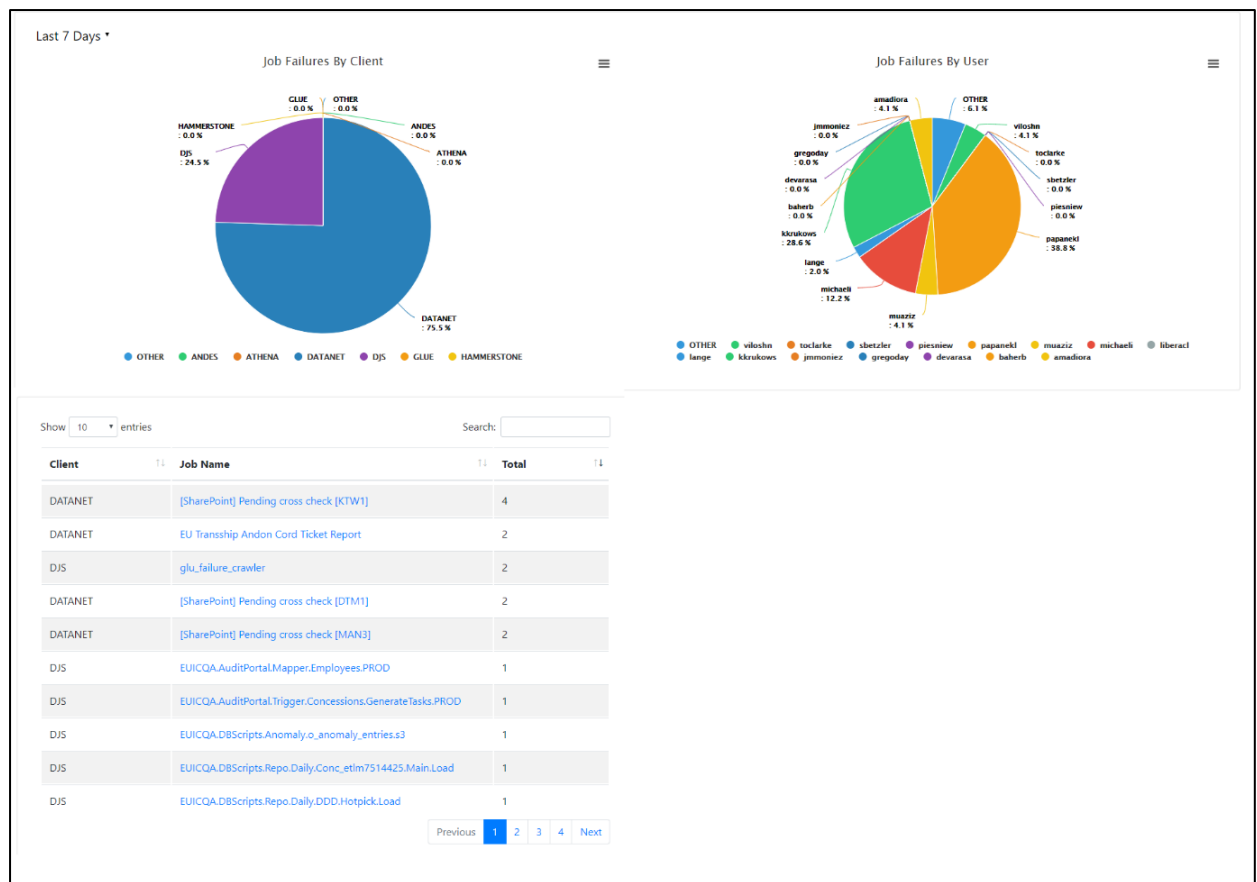


Figure 14 Breakdown of Job failures by client and user

Redshift Queue Management

EU ICQA HEIMDALL
ADMIN

REDSHIFT QUERIES - Last updated on 2019-09-10

Query Date From:
 Query Date To:

Scheduled Job:
 Job Run status:

Order By:
 Min:
 Max:

DB User:
 Number Of Entries:

Show 10 ▾ entries Search:

Snapshot Date	User	Db User	Job Id	Job Run Id	Job Run Status	Query	Queue Time/min	Execution time/min	Total Time/min	Start Time
2019-09-06	krukows	euicqars_rs_ro	17847293	5282528881	Deleted	CREATE /*** jobrun_id:528252	75.6	60.0	135.6	2019-09-06 06:25:36
2019-09-06	mlipows	euicqars_rs_ro	15942593	5281915775	Success	SELECT /*** jobrun_id:528191	67.0	57.6	124.6	2019-09-06 07:46:58
2019-09-05	hubble_ddl	hubble_ddl				/*** Hubble: User-krukows, S	0.0	120.0	120.0	2019-09-05 04:11:48
2019-09-06	krukows	euicqars_rs_ro	17835005	5281878101	Deleted	CREATE /*** jobrun_id:528187	110.3	0.3	110.7	2019-09-06 06:50:18
2019-09-06	krukows	euicqars_rs_ro	17799996	5281870121	Deleted	CREATE /*** jobrun_id:528187	109.9	0.7	110.6	2019-09-06 06:47:17
2019-09-06	krukows	euicqars_rs_ro	17835004	5281873703	Deleted	CREATE /*** jobrun_id:528187	110.3	0.2	110.5	2019-09-06 06:50:53

Figure 15 Data Table displaying queries based on their query and execution times

In Figure 15, you can see a table displaying the queries with highest queue or execution or total time depending on the user's choice. The user can also narrow the results using the available filters.

Figure 16 below, shows two charts which are useful to analyse the execution of the queries over the last 7 days.

The first chart is a dual axis chart describing the mean queue, execution and total time on one axis and the total number of queries on the second one.

The second chart is a boxplot showing the min, max, median, upper quartile and lower quartile execution times. In a boxplot diagram, the outliers can be easily identified.

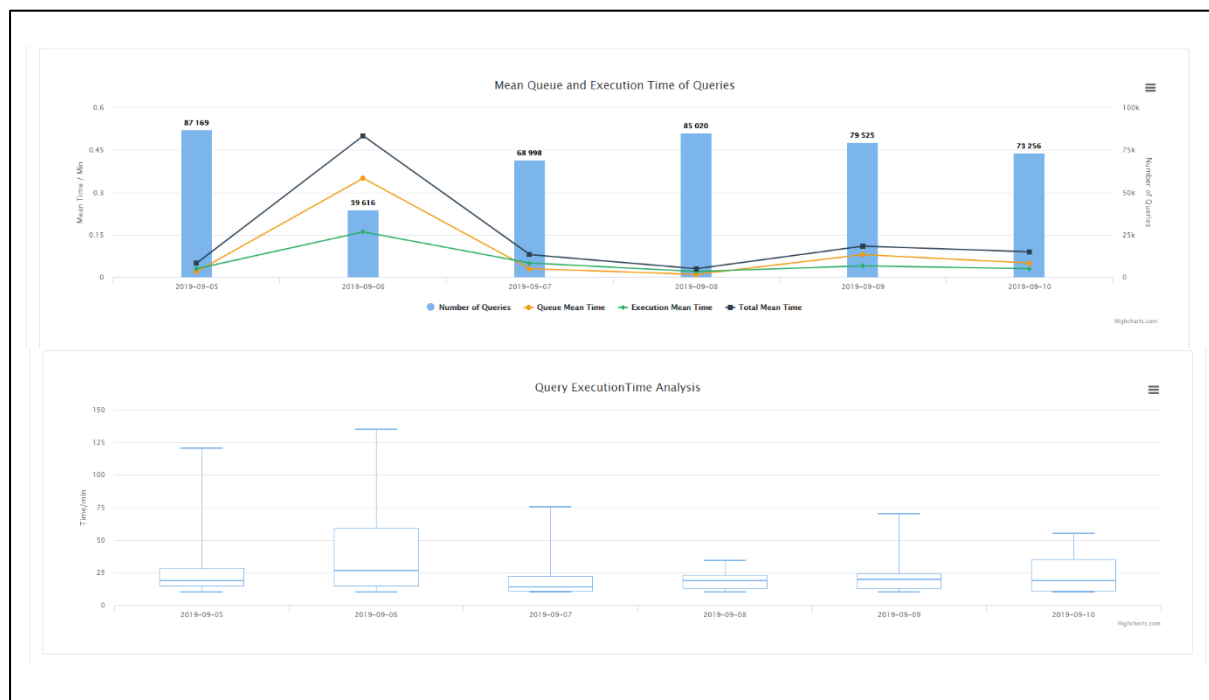


Figure 16 Charts showing metrics on query execution times.

User Visits

The user visits dashboard compares the number of user visits and distinct users for each portal. Users can also view the data grouped by the FC, site type and operational area. This dashboard helps the team to have an overview of which portals are most visited to enhance our services. It also allows us to identify whether there are some FCs which have difficulty to use the portals and reach out to them to investigate why this is the case.



Figure 17 User Visits Dashboard

Data Control Strategies

In the data control strategies main page, users can view existing rules and execute the rules by clicking on the run icon. They can view and edit a rule by clicking on the “eye” icon. They can see details like last run status and the SIM id to track the issue.

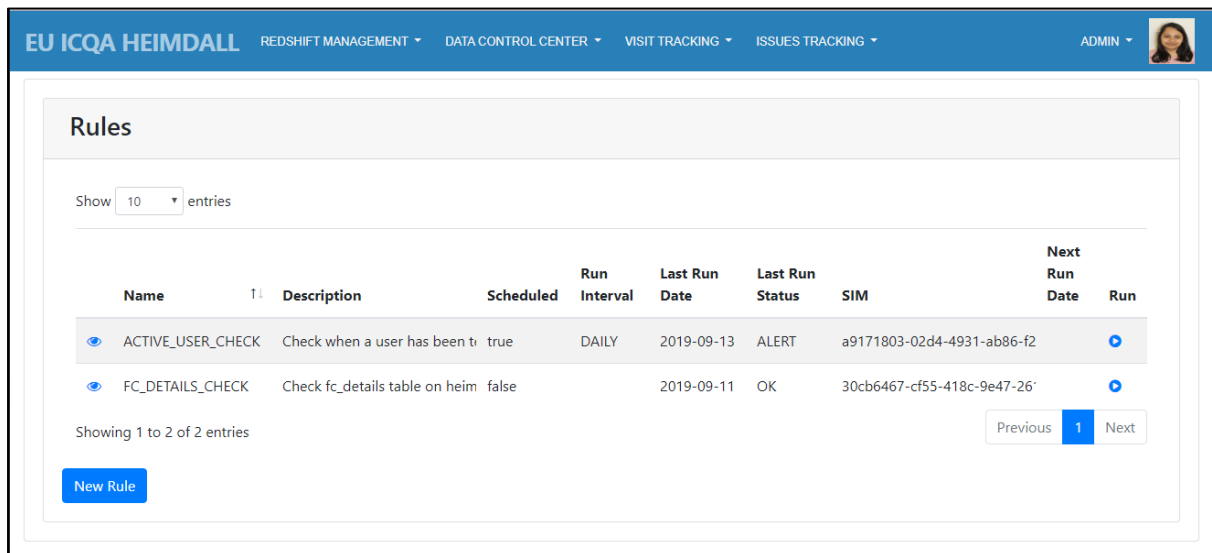


Figure 18 Page to view and run rules

Figure 19 shows that a SIM is successfully created using the SIM API when a rule fails the control check and is in alert state.

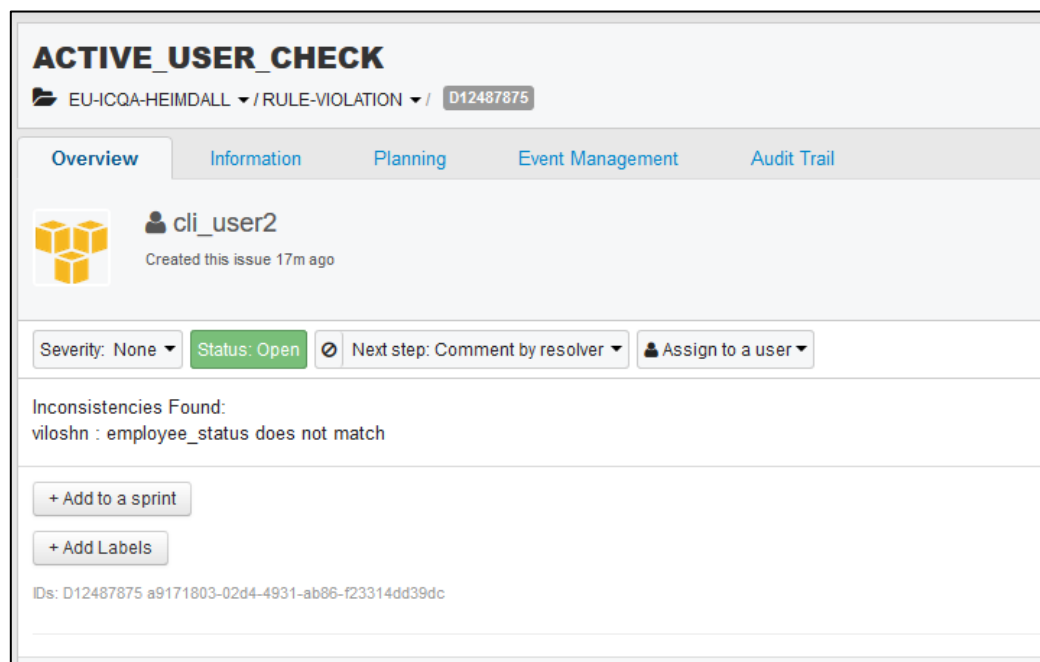


Figure 19 Creation of a SIM issue when a rule fails

The same form is used to create and edit a rule. When editing, the fields are pre filled with the values. The database should first be configured in Heimdall connections view to use it in the screen below.

Edit Rule

RULE NAME: FC_DETAILS_CHECK

DESCRIPTION: Check fc_details table on heimdall and redshift

PRIMARY DATABASE: HEIMDALL

SECONDARY DATABASE: EU_ICQA_REDSHIFT

PRIMARY QUERY: SELECT warehouse_id as id, region_id, site_type FROM fc_details WHERE site_definition = 'AMZN_FC'

SECONDARY QUERY: SELECT warehouse_id as id, region_id, site_type FROM public.fc_details WHERE site_definition = 'AMZN_FC'

OUTPUT TYPE: RECORD LIST

SCHEDULED: YES

RUN INTERVAL: DAILY

Save Cancel

Figure 20 Creation of a new rule

Admin Views

The admin views are used to add and modify parameters. Figure 21 shows the Portal View where you can view existing portal and add a new portal. The portal parameters are used to by the user visit module(both rake task and dashboard). When you click on the Last Import column, details about the last import will be displayed.

Portals

Show 10 entries

Name	Database	Table Name	Id Field Name	User Field Name	Path Field Name	Excluded Paths	Active	Last Import
star	STAR PROD	uservisits	id	remote_user	path		Yes	OK
home	HOME PROD	uservisits	id	username	routing		Yes	OK
pacman	PACMAN PROD	uservisits	id	username	routing		Yes	OK
repo	REPO PROD	uservisits	id	remote_user	path		Yes	OK
tsportal	TS PORTAL PROD	uservisits	id	username	routing		Yes	OK
inbound	INBOUND PORTAL	uservisits	id	remote_user	path	/landing/header	No	OK
heimdall	HEIMDALL	heimdall_user_visits	id	username	path		No	KO

Showing 1 to 7 of 7 entries

Previous 1 Next

New Portal

Figure 21 Databse containing portal details

The following screen shows the teams and users details. I can be used to add or modify or delete a team or user. Clicking on the Bulk Update button will invoke the “Import Users” rake task. This is mainly useful when a new team is created and we have to import all the developers in that team.

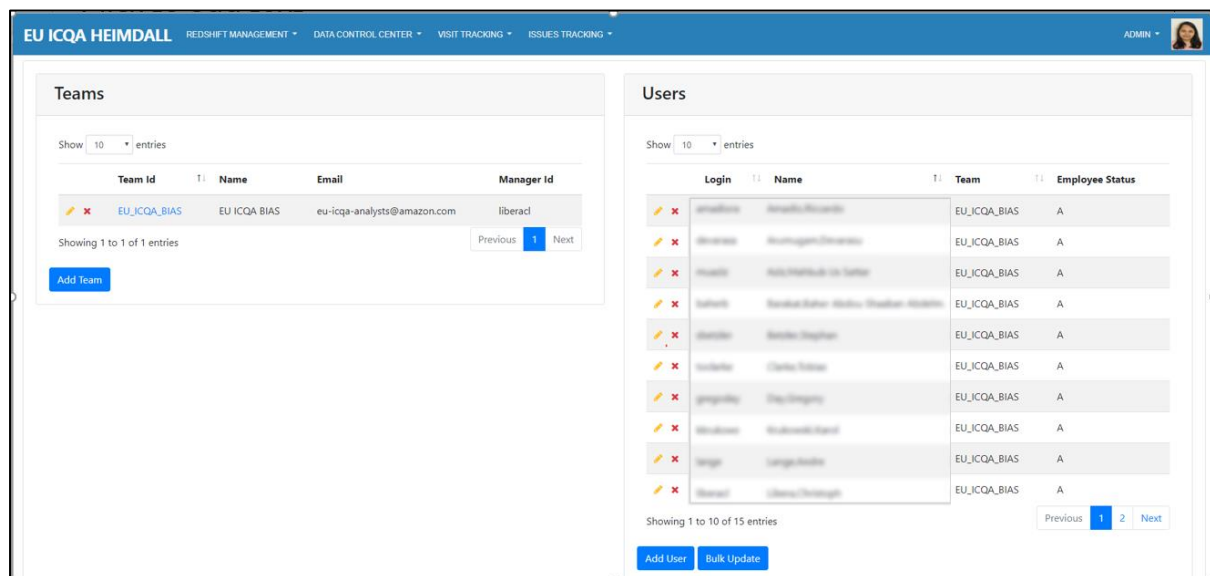


Figure 22 View to consult Teams and Users details

The connections view displays the connection parameters to different databases. The parameters are used by user visit rake task and data control strategies.

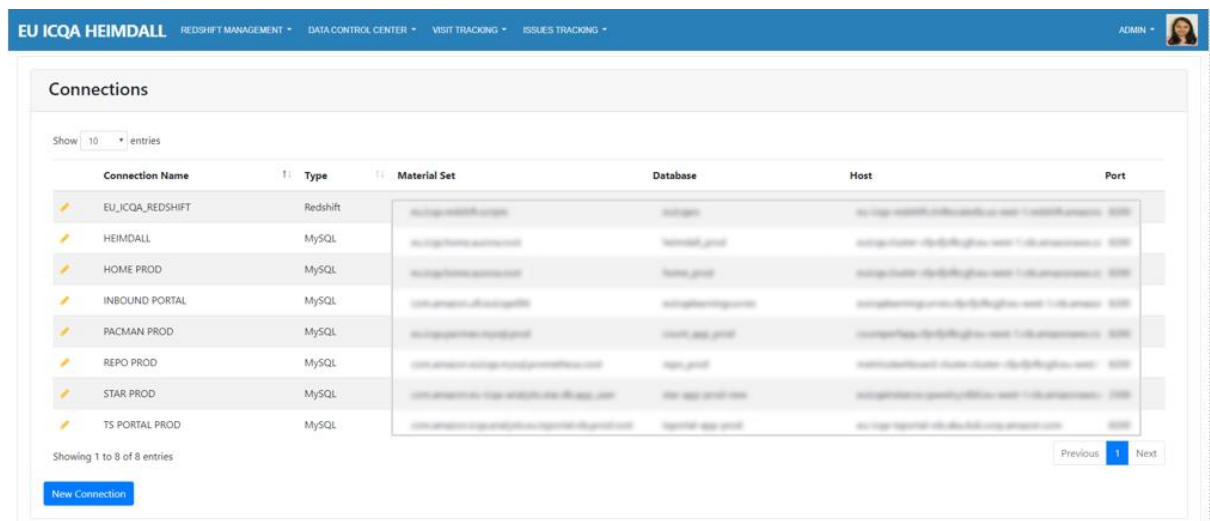


Figure 23 View to display and edit database connections

Finally, Figure 24 shows the Alarms view. Users can create a new alarm by specifying the datanet group and alarm name. The Datanet API is called to create the alarm.

EU ICQA HEIMDALL
REDSHIFT MANAGEMENT
DATA CONTROL CENTER
VISIT TRACKING
ISSUES TRACKING

Alarms

Show
10
entries

Alarm Name	Alarm Id	Group Name
heimdall_eu_icqa_virt_group	5046442	eu-icqa-virt-group
heimdall_eu_icqa_analysts_tickets	5047092	eu-icqa-analysts-tickets
heimdall_eu_icqa_rs	5104438	eu-icqa-rs
heimdall_uk_icqa	5126564	uk_icqa

Showing 1 to 4 of 4 entries

Previous
1
Next

New Alarm

Figure 24 Display and create new Datanet alarms

CHALLENGES AND SOLUTIONS

Import of redshift queries

Initially a rake task was executed daily at a scheduled time to extract redshift queries from the redshift table and import it into Heimdall portal. However, often, the ETL job preparing the data was stuck in the waiting queue because the resources were not available. Hence, when the rake task started execution, no records were available to import.

To overcome this issue, a lambda function is used to load the Heimdall table. When the ETL job has finished execution and dumped the data in an S3 bucket, the lambda will be invoked to populate the Heimdall database. Since the lambda function is event triggered, it will run only when the data is available and we are sure to never miss an import.

User Visit rake task

During the import, before inserting user visit logs into Heimdall database, the location of the user had to be retrieved and stored for later processing. Since this data is not available in the

original tables, a join was made with the employees table. The following problems were encountered:

- Querying the table for each user visit record was time consuming. The “IN” operator could be used to avoid several database calls but the table contains around one million rows so the “IN” operation was even more time consuming.
- The table contains only employee details working in FCs in the EU region only. We could not match employees in CORP offices or outside EU region.
- The employees table had to be updated daily to cater for employees who are changing location.

To counter the above issues, I decided to use LDAP API to retrieve user location. The LDAP API contains information about all employees and is reliable. To further optimise the script,, I used a hash map. The key is the user login and the value is the location. For each record, I first checked the map. If the user login is already present, I take the location from the map. If the user is not present, I make a call to LDAP API and store the login-location pair in the map avoiding an API call for the same user. This optimised script took around 100 seconds less (28.3 s) than the unoptimised one (125.3 s) for the same dataset of 6442 records.

```
(1.0ms) BEGIN  
(1.1ms) COMMIT  
-----archiving old records-----  
Portal Load (1.1ms) SELECT `portals`.* FROM `portals` W  
DatabaseConnection Load (1.0ms) SELECT `database_connec  
>>>>>>>end at: 2019-09-13 13:56:36 +0000<<<<<<<<<<<<  
>>>>>>>Duration: 125.289976203<<<<<<<<<<<<
```

Figure 25 Execution time without optimisation

```
(1.8ms) BEGIN  
(1.5ms) COMMIT  
-----archiving old records-----  
Portal Load (2.4ms) SELECT `portals`.* FROM `portals` W  
DatabaseConnection Load (1.4ms) SELECT `database_conne  
>>>>>>end at: 2019-09-13 13:52:15 +0000<<<<<<<<<<<<  
>>>>>>Duration: 28.258564141<<<<<<<<<<<<
```

Figure 26 Execution time with hash map optimisation

Loading time of Redshift dashboard

Although the redshift table stores data for the last 7 days only, it still contains around 400,000 rows. This slows the processing time of the Redshift queue dashboard. Since calculating the median, upper quartile and lower quartile, using SQL query is not efficient, all the records have to be fetched and looped through to calculate those metrics. This further slows down the processing.

To increase the speed, I have implemented an in-memory cache. The in-memory cache is a simple caching mechanism where the key is the md5 hash of the query and the value is the already calculated metrics. The in-memory cache is stored as a global variable. Each time a user accesses the page, the application checks if the hash of the same query is present in the cache. If yes, it loads the results from the cache instead of fetching the whole dataset. The cache does not take up much space because we are not storing the records but the already calculated metrics instead. The in-memory cache can take up to 2MB which will not have a big impact the whole application.

Caching is possible in this case because the data is not live and is updated daily. The cache is cleared every day to refresh to new values. By using this caching mechanism, the processing time of 400k records dropped from 20,000 ms to around 10 ms.

7. ASSESSMENT

This chapter assesses the work done and highlights the skills acquired during the internship.

ASSESSMENT OF PROJECT

At the end of the internship, a working software was successfully delivered and released on production. All the initial objectives have been met and the tasks completed. I was even able to include the Data Control Strategies module. Although it only caters for simple rules for the time being, it is the base for future enhancements and additional features.

The job failure notifications have helped the team to address the issues faster and more efficiently. We are able to identify the jobs that are failing more frequently and fix the root causes. As shown in the line chart below, the number of job failures has decreased and follows a more stable trend during the last 2 weeks compared to the last month. I did not include data for July because the functionality was still undergoing tests and modifications. Hence, the data for July is not reliable.

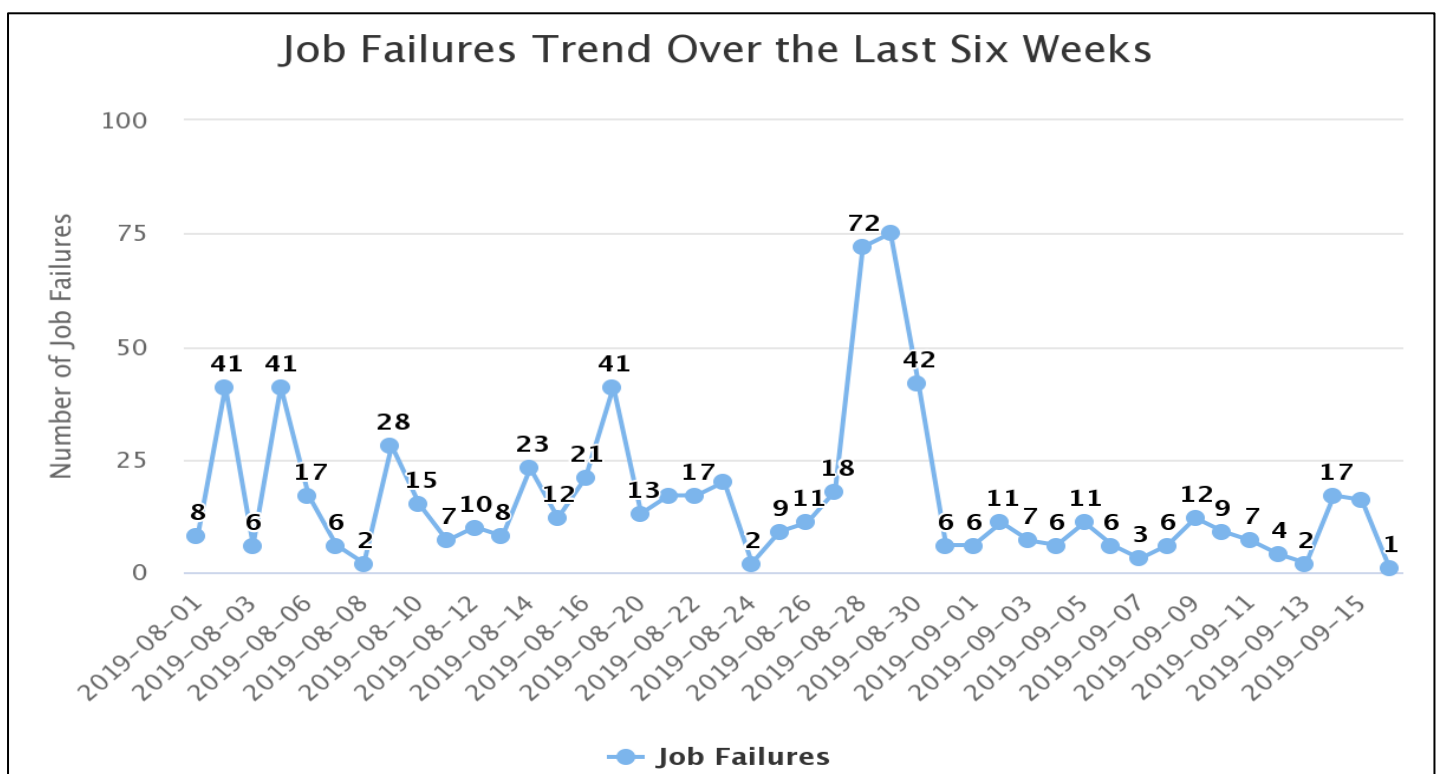


Figure 27 Job Failure Trend over the past two months

Unfortunately, the usage metrics for the platform are not available as at submission of the report since the final handing over is not yet done. However, positive feedback was received during the tech call demos. The team was in favour of the tool and said they will definitely use it. They found it useful to track progress and evaluate their performance as well as monitor that of our different applications.

SKILLS ACQUIRED

This internship was packed with learning opportunities. I was exposed to innovative cloud technologies and services.

I had hands-on experience on a new web development framework (Ruby on rails) and with user interface design using bootstrap.

I also had the chance to explore big data services like AWS Redshift and the ETL Manager. I learnt how to write and optimise complex queries to efficiently extract data and process the data. The highlight of the internship was to discover serverless architecture and work with AWS lambda and SQS queue.

During tech calls, someone from the team would often talk on an interesting topic. The following topics covered during those 6 months: Ruby Active records, AWS lambda and Machine Learning. I was introduced to AWS Eider, a Python Notebook used mainly for Machine Learning. It supports multiple libraries including sk-learn and data visualisation libraries.

Working on this project also enabled me to improve my analytical skills and pushed to examine my codes and to constantly optimise instead of just providing working software. I also learnt that the best codes are not always the most complex ones. Sometimes, the best solutions are the simplest ones.

On top of the technical skills, this internship also improved my soft skills. The presentations and demos helped me to enhance my communication skills in a real working environment. At first, it was a bit tough to interact and ask questions because it was a different environment from what I am used to. However, after 2-3 weeks I gain in confidence and participated more actively.

Amazon has a program, the Women's Internship Network in Operations (WIN-In-Ops) which is a leadership and networking program for women interns. The goals of the program are to empower new female interns to take charge of their career, to support their professional growth, and to give them the opportunity to learn to handle challenges women can face in the workplace. I had the chance to participate in a 3 day workshop during which I got many useful tips on leadership and networking.

HOW DID THE 2IS MASTER PROGRAM HELP DURING THIS INTERNSHIP?

The knowledge gained and skills acquired through this internship are things that complemented and enhanced the education I obtained at UT1. Although, all the courses we had at UT1 contributed in some way or another to the successful completion this internship, the following courses were more useful because they are more closely related to the subject of and tasks carried during the internship.

Advanced Programming and Computational Intelligence

Advanced programming introduced me to the good practices of programming and data structures. The material covered during CI course (search algorithms and code complexity analysis) helped me to analyse my code and come up with optimise solution.

Business Intelligence and Innovative Data Management

The BI class strengthen my knowledge on Relational Databases and SQL. Moreover, it initiated me to business intelligence and the importance of data visualisation, which was a major component of this project. The Innovative Data Management course helped me to manage and process large volumes of data.

Project Management and Innovative Software Management

The different projects and role-play assignments gave me a good grasp on agile methodologies and scrum. I could easily understand and adapt to the working environment and manage my tasks.

FUTURE WORKS

The Heimdall portal holds potential for a lot of additional features and enhancements.

- As mentioned in the needs analysis section, ETL job often have long queue time because of unavailability of resources. An algorithm should be designed to analyse the root causes and backtrack to resources that are initiating this issue.
- Around 70k – 80k queries are executed on the redshift query every day. We could extract the table names to analyse which tables are most impacted and where can we optimise in terms of usage of index or partitioning.
- Dive deeper in the job failures to automatically extract the logs and root causes. This will further reduce resolution time.
- Enhance the data control strategies modules by defining new strategies and algorithms that can be used to identify errors in reports automatically. For example: unrealistically high/low values, quick changes, missing values. Machine Learning techniques can be used to achieve this.
- Another improvement would be to schedule the execution of the rules so that they are automated.

8. CONCLUSION

Overall, I would describe my internship as a positive and enriching experience. I had to opportunity to work with cutting edge technology. Not only did I gain technical skills but I also met some fantastic people and learnt about a new culture. Although there are some room for improvement and additional features, I am satisfied to have delivered a good and working tool that will be useful to the team. I am forever grateful to all those who accompanied me during this journey. This internship will definitely be a stepping stone in my career and I look forward to applying the skills acquired in my future projects.

9. REFERENCES

1. amazon.jobs. (2019). *Amazon's global career site*. [online] Available at: <https://www.amazon.jobs/en/principles> [Accessed 1 Sep. 2019].
2. Amazon Web Services, Inc. (2019). *Amazon Simple Queue Service (SQS) | Message Queuing for Messaging Applications | AWS*. [online] Available at: <https://aws.amazon.com/sqs/> [Accessed 1 Sep. 2019].
3. Docs.aws.amazon.com. (2019). *Getting Started with Amazon Redshift - Amazon Redshift*. [online] Available at: <https://docs.aws.amazon.com/redshift/latest/gsg/getting-started.html> [Accessed 1 Sep. 2019].
4. Sqs-public-images.s3.amazonaws.com. (2019). [online] Available at: http://sqs-public-images.s3.amazonaws.com/Building_Scalabale_EC2_applications_with_SQS2.pdf [Accessed 1 Sep. 2019].
5. Amazon Web Services, Inc. (2019). *Amazon Athena — Serverless Interactive Query Service - AWS*. [online] Available at: <https://aws.amazon.com/athena/> [Accessed 7 Sep. 2019].
6. Amazon Web Services, Inc. (2019). *AWS Glue - Amazon Web Services*. [online] Available at: <https://aws.amazon.com/glue/> [Accessed 7 Sep. 2019].
7. Medium. (2019). *What the heck is Rake?*. [online] Available at: <https://medium.com/craft-academy/what-the-heck-is-rake-b44d4210922b> [Accessed 15 Sep. 2019].
8. Ruby on Rails Guides. (2019). *Active Record Basics — Ruby on Rails Guides*. [online] Available at: https://guides.rubyonrails.org/active_record_basics.html [Accessed 15 Sep. 2019].
9. Amazon Web Services, Inc. (2019). *AWS Lambda*. [online] Available at: <https://aws.amazon.com/lambda/> [Accessed 15 Sep. 2019].
10. Moris, S. (2019). *4 Key Things You Need to Know About Ruby on Rails*. [online] Available at: <https://skillcrush.com/2015/01/29/13-ruby-rails/> [Accessed 15 Sep. 2019].
11. Castello, J. (2019). *What is Ruby on Rails & Why Is It Useful?* [online] Available at: <https://www.rubyguides.com/2018/10/what-is-ruby-on-rails/> [Accessed 15 Sep. 2019].
12. Haml. (2019). *Haml - About* [online] Available at: <http://haml.info/about.html> [Accessed 15 Sep. 2019].